Shrawani Khandagale
Roll no.-69

Practical No.-8

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

# Load Iris dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['Target'] = iris.target

# Display first five rows
df.head()
```

```
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width
(cm)  \
0                5.1               3.5                1.4
0.2
1                4.9               3.0                1.4
0.2
2                4.7               3.2                1.3
0.2
3                4.6               3.1                1.5
0.2
4                5.0               3.6                1.4
0.2

   Target
0       0
1       0
2       0
3       0
4       0
```

```
X = iris.data
y = iris.target

# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42
)
```

```
print("Training Samples:", len(X_train))
print("Testing Samples:", len(X_test))
```

```
Training Samples: 105
Testing Samples: 45
```

```
knn = KNeighborsClassifier(n_neighbors=3)

knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report")
print(classification_report(y_test, y_pred))
```

```
Accuracy: 1.0

Confusion Matrix
[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]

Classification Report
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       1.00      1.00      1.00        13
           2       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```

```
k_values = range(1, 21)

accuracy_list = []

for k in k_values:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)

    prediction = model.predict(X_test)

    acc = accuracy_score(y_test, prediction)
```

```
    accuracy_list.append(acc)

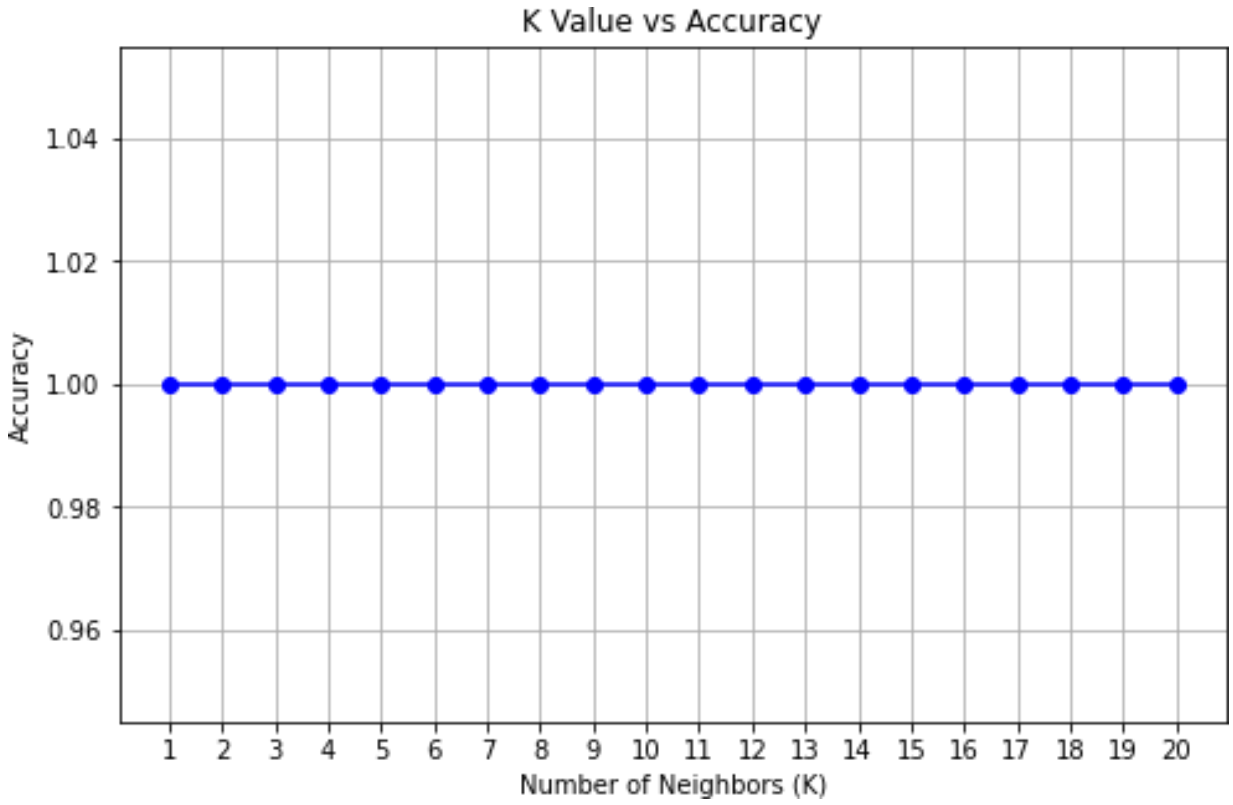
plt.figure(figsize=(8,5))

plt.plot(k_values, accuracy_list,
         marker='o',
         linestyle='-',
         color='blue')

plt.title("K Value vs Accuracy")
plt.xlabel("Number of Neighbors (K)")
plt.ylabel("Accuracy")

plt.xticks(k_values)
plt.grid(True)

plt.show()
```




```
best_k = k_values[np.argmax(accuracy_list)]
best_accuracy = max(accuracy_list)

print("Best K:", best_k)
print("Highest Accuracy:", best_accuracy)
```

```
Best K: 1
Highest Accuracy: 1.0
```